



UNIVERSITY OF GHANA

(All rights reserved)

**MPhil/MSc. COMPUTER SCIENCE & MPhil/MSc. DATA SCIENCE, FIRST SEMESTER
EXAMINATIONS: 2025/2026**

DEPARTMENT OF COMPUTER SCIENCE

CSCD602: ADVANCED SOFTWARE ENGINEERING (3 CREDITS)

INDIVIDUAL PROJECT-BASED EXAMINATION

1. *Assessment: Individual Software Engineering Capstone Examination*
2. *Examination Duration: 48 Hours*
3. *Total Marks: 50 Marks*
4. *Submission Platform: Sakai LMS*
5. *Assessment Mode: Individual*

EXAMINATION QUESTION

Within the 48-hour examination period, independently identify a software problem or opportunity of your choice and develop a functional, deployable software application using an appropriate software engineering lifecycle.

Your work must demonstrate the application of requirements engineering, software effort estimation, analysis, design, implementation, testing, deployment, documentation, maintenance and future software evolution.

Particular attention must be given to software effort estimation during requirements analysis and the identification, prioritisation and management of technical debt during implementation, testing and future maintenance.

The project must be appropriately scoped for completion within 48 hours. You are not expected to develop a large commercial system; rather, you are expected to demonstrate sound and systematic Advanced Software Engineering practice.

PART A: EXAMINATION INSTRUCTIONS

1. Individual Work

- This is an individual end-of-semester examination.
- You must independently select, analyse, design, implement, test and deploy your software project.
- Collaboration on the development of another student's solution is not permitted.
- You may discuss general software engineering concepts but must not share your project source code, SRS, design documents or completed solution with another student.

2. Project Selection

- Select a realistic software problem or business opportunity.
- The project may be related to business, education, healthcare, finance, agriculture, transportation, entertainment, public services or another appropriate domain.
- Clearly define the problem and intended users.
- Do not select a project that is too large to implement within 48 hours.
- The project must contain enough functionality to demonstrate the software engineering principles covered in the course.

3. Requirements Before Development

Do not begin implementation immediately.

First:

1. Define the problem.
2. Identify stakeholders/users.
3. Gather and analyse requirements.
4. Define functional and non-functional requirements.
5. Prioritise requirements.
6. Estimate software effort.
7. Define the scope of the system to be completed within the 48-hour period.

Your requirements and effort estimation should guide your implementation decisions.

4. Software Effort Estimation

Select and apply an appropriate effort-estimation technique, such as:

- Function Point Analysis
- Use Case Points
- COCOMO/COCOMO II
- Story points
- Expert estimation
- Another justified estimation approach

Explain:

- Why the technique was selected
- Estimated effort
- Estimated person-hours
- Estimated development duration
- Assumptions
- Constraints
- How the estimation influenced your project scope

5. System Design

Develop appropriate design artefacts before or alongside implementation.

Depending on your project, these may include:

- System architecture
- Use-case diagram
- Class diagram
- Sequence diagram
- Activity diagram
- ER/database diagram
- Component diagram
- UI wireframes
- Data-flow diagram

You do **not** need to produce every possible diagram. Select the diagrams that best communicate your system.

6. Implementation

Develop a functional software application, not merely:

- A proposal
- A requirements document
- A UI mock-up
- A static website
- A non-functional prototype

The application should implement the major prioritised requirements.

Where applicable, include:

- Front-end
- Back-end
- Database
- Authentication
- Authorisation
- API integration
- Input validation
- Error handling
- Security controls
- Responsive interface

7. Technical Debt

You must explicitly consider technical debt during the project.

Document technical debt introduced because of:

- 48-hour time constraints
- Design shortcuts
- Simplified architecture
- Incomplete refactoring
- Temporary solutions
- Limited testing

- Code duplication
- Dependency choices
- Security compromises
- Incomplete documentation

For each significant technical debt item, identify:

Debt → Cause → Impact → Priority → Proposed Resolution

You should clearly distinguish between technical debt that is:

- Acceptable temporarily
- Scheduled for future resolution
- Critical and requiring immediate attention

8. Testing

Develop and execute appropriate tests.

At minimum, provide evidence of relevant:

- Functional testing
- Unit testing, where applicable
- Integration testing, where applicable
- System testing
- User acceptance testing

Where appropriate, also consider:

- Security testing
- Usability testing
- Performance testing

Document:

- Test case
- Expected result
- Actual result
- Pass/fail
- Defects identified
- Corrective action

9. Deployment

The application must be deployed and accessible online.

You may use:

- Vercel
- Netlify
- Render
- GitHub Pages, where appropriate
- Cloud platforms
- Another suitable hosting platform

Provide the examiner with:

- Live application URL
- Admin URL, if applicable
- Test-user credentials
- Admin credentials, if applicable
- Source-code repository

10. Documentation

Prepare a consolidated project document covering:

1. Project title
2. Problem statement
3. Aim and objectives
4. Stakeholders
5. Requirements analysis
6. SRS
7. Software effort estimation
8. System analysis
9. System design
10. Implementation
11. Testing
12. Technical debt
13. Deployment
14. User manual
15. Maintenance strategy
16. Future evolution
17. Limitations
18. Conclusion
19. References

11. Maintenance and Future Evolution

Explain how the software should be maintained after deployment.

Consider:

- Corrective maintenance
- Adaptive maintenance
- Perfective maintenance
- Preventive maintenance
- Security updates
- Dependency updates
- Performance improvements
- Scalability
- New features
- User feedback
- Technology changes

Include a technical debt repayment plan showing how identified technical debt will be addressed in future versions.

PART B: RECOMMENDED 48-HOUR WORK PLAN

Students are encouraged to manage their examination time systematically.

Phase 1 — Planning & Requirements

Recommended: Hours 1–6

- Select project
- Define problem
- Identify stakeholders
- Gather requirements
- Define SRS
- Prioritise requirements
- Conduct effort estimation
- Define project scope

Phase 2 — Analysis & Design

Recommended: Hours 7–12

- Design architecture
- Develop appropriate UML diagrams
- Design database
- Design interfaces
- Define technology stack
- Identify potential technical debt

Phase 3 — Implementation

Recommended: Hours 13–32

- Develop core functionality
- Implement database
- Implement user interface
- Develop backend
- Integrate APIs/services
- Implement authentication/security where required
- Conduct continuous testing

Phase 4 — Testing & Refinement

Recommended: Hours 33–38

- Execute test cases
- Fix critical defects
- Conduct usability checks
- Review security
- Document technical debt
- Refactor critical areas where feasible

Phase 5 — Deployment

Recommended: Hours 39–42

- Deploy application
- Verify production environment

- Test live application
- Verify database/API connections
- Prepare test credentials
- Verify source-code repository

Phase 6 — Documentation & Final Submission

Recommended: Hours 43–48

- Complete project documentation
- Complete user manual
- Finalise testing report
- Finalise technical debt plan
- Finalise maintenance/evolution plan
- Verify all links
- Package files
- Submit through SAKAI LMS

This is a recommended schedule, not a mandatory timetable. Students may organise the 48 hours according to their project requirements.

PART C: FINAL SUBMISSION REQUIREMENTS

Each student must submit ONE ZIP FILE through the SAKAI LMS containing:

StudentID_ProjectName/

```

|— Project_Documentation.pdf
|— SRS.pdf
|— Testing_Report.pdf
|— Technical_Debt_Plan.pdf
|— User_Manual.pdf
|— Deployment_and_Source_Links.txt
|— Supporting_Files/

```

The files may be combined into one comprehensive PDF if preferred, provided all required sections are clearly identified.

Deployment_and_Source_Links.txt

Include:

Student Name:

Student ID:

Project Title:

Live Application:

Admin URL:

Test Username:

Test Password:

Admin Username:

Admin Password:

Source Code Repository:

PART D: FINAL SUBMISSION CHECKLIST

Before submitting to SAKAI, confirm that you have:

- Defined a realistic software problem.
- Identified stakeholders and users.
- Completed requirements analysis.
- Developed an SRS.
- Estimated software effort.
- Justified the estimation technique.
- Designed the system.
- Implemented a functional application.
- Tested the application.
- Documented test results.
- Identified technical debt.
- Proposed technical debt resolution strategies.
- Deployed the application.
- Tested the live deployment.
- Prepared a user manual.
- Prepared a maintenance strategy.
- Prepared a future evolution plan.
- Provided the source-code repository.
- Verified all URLs and credentials.
- Included your name, student ID and project title.
- Submitted all required files through SAKAI.

PART E: ASSESSMENT SCHEME — 50 MARKS

Assessment Component	Marks
Requirements Engineering & SRS	7
Software Effort Estimation	5
System Analysis & Design	6
Implementation & Functionality	10
Testing & Quality Assurance	5
Technical Debt Identification & Management	6
Deployment & Accessibility	3
Documentation & User Manual	3
Maintenance & Future Evolution	3
TOTAL	50

PART F: EXAMINATION RULES

1. This is an individual examination.
2. The examination period is 48 hours.
3. The application must be developed within the examination period.
4. The project must be appropriately scoped for the available time.
5. Students must demonstrate their own understanding of the submitted work.
6. All external resources, libraries, frameworks, APIs, datasets and third-party components must be appropriately acknowledged.
7. Source code must be maintained in a repository and made available to the examiner where required.

8. The deployed application must remain accessible for grading.
9. Students are responsible for verifying that their submitted links and credentials work.
10. The examiner may conduct an individual viva voce/demo to verify authorship, understanding and implementation.
11. Students may be asked to explain their requirements, effort estimation, architecture, implementation decisions, testing strategy and technical debt.
12. Any significant reuse of previously submitted academic work must be disclosed and appropriately referenced.
13. Academic integrity requirements apply throughout the examination.

Final Examination Principle

You are not being assessed simply on whether you can produce a working application in 48 hours. You are being assessed on whether you can demonstrate disciplined Advanced Software Engineering practice under a realistic time constraint.

Your submission should therefore demonstrate the complete lifecycle:

Requirements → Effort Estimation → Analysis → Design → Implementation → Testing → Technical Debt Management → Deployment → Documentation → Maintenance → Future Evolution